# 01 DFS

Depth First Search | reachability, traversal, flood fill

## 01 WHAT IS IT?

Explore one branch fully before returning. Each vertex is visited once; recursive calls form the search stack.

## 02 WHEN TO USE

- Can A reach B? Explore reachable nodes.
- Traverse a component; flood-fill a region.
- DFS tree, entry/exit times, cycle detection.

## 03 CORE IDEA

1. Mark the current node before exploring.
2. Process it; inspect each neighbor.
3. Recurse only into unvisited neighbors.
4. Return when this branch is exhausted.

## 04 MEMORY TRICK

**Go deep, then come back.**

## 05 IMPORTANT DATA STRUCTURES

- **Adjacency list** - enumerate neighbors.
- **visited[]** - stop repeat visits/cycles.
- **Call stack** - remember unfinished branches.

## 06 OPERATIONS / ALGORITHM FLOW

START: set visited[] = false
↓
Choose source s; mark s
↓
Process u; inspect each neighbor v
↓
Unvisited v? Recurse; otherwise skip
↓
No more neighbors? Return to parent
↓
END: source component explored

## 07 TIME & SPACE COMPLEXITY

| Operation / Case | Time | Space |
|---|---|---|
| Adjacency list | O(V + E) | O(V) |
| Adjacency matrix | O(V^2) | O(V) |

**INTERVIEW: O(V + E) time, O(V) space.** A single-source search touches only reachable edges; array initialization is O(V).

## 08 CODE SKELETON

Class fields: **List<Integer>[] g**, **boolean[] seen**. Initialize seen to false; use **java.util.***.

```
void dfs(int u) {
    seen[u] = true;
    for (int v : g[u])
        if (!seen[v]) dfs(v);
}
// One source:
dfs(s);
// Whole graph:
for (int u = 0; u < n; u++)
    if (!seen[u]) dfs(u);
```

## 09 KEY VARIABLES TO REMEMBER

- **u** - current node; **v** - neighbor.
- **s** - source; **seen** - persistent visited state.
- **g, n** - adjacency list and vertex count.

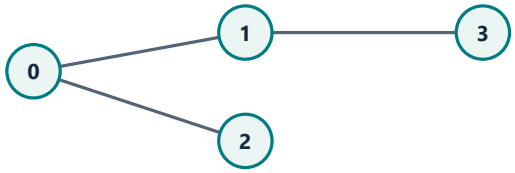
## 10 COMMON MISTAKES

- Marking after recursion: cycles recurse forever.
- One source misses disconnected components.
- Deep recursion can overflow Java's stack.
- Visited alone does not detect directed cycles; add an active-stack flag.

## 11 RECOGNITION PATTERN

**If the question says... → Think...**
"Reach all nodes from s" → DFS or BFS.
"Connected land cells" → flood-fill DFS.

## 12 ONE TINY EXAMPLE




dfs(0): **0, 1, 3, 2** with neighbors in shown order. Return from 3 to 1 to 0, then visit 2.

# 02 BFS

Breadth First Search | shortest hops, levels, simultaneous spread

## 01 WHAT IS IT?

Explore nodes in distance layers with a FIFO queue. In an unweighted graph, first discovery gives the minimum number of edges from the source.

## 02 WHEN TO USE

- Minimum hops / moves; all edges cost 1.
- Level order, nearest exit, shortest grid path.
- Spread from several starting cells at once.

## 03 CORE IDEA

1. Set all distances to -1; source to 0.
2. Enqueue the source immediately.
3. Pop u; discover each unseen neighbor v.
4. Set dist[v] = dist[u] + 1; enqueue v.

## 04 MEMORY TRICK

**Level by level. Mark when you enqueue.**

## 05 IMPORTANT DATA STRUCTURES

- **Queue** - process earliest discovery first.
- **dist[]** - distance; -1 means unseen.
- **parent[]** (optional) - rebuild a path.

## 06 OPERATIONS / ALGORITHM FLOW

START: dist[] = -1
↓
Set dist[s] = 0; enqueue s
↓
Queue empty? END
↓
Dequeue u; inspect neighbors v
↓
dist[v] == -1? Set distance, enqueue
↓
Repeat until queue is empty

## 07 TIME & SPACE COMPLEXITY

| Operation / Case | Time | Space |
|---|---|---|
| Single / multi-source | O(V + E) | O(V) |
| Grid, 4 neighbors | O(N) | O(N) |

**INTERVIEW: O(V + E) time, O(V) space.** Multi-source: seed all sources at distance 0.

## 08 CODE SKELETON

Assume initialized adjacency list **g**, vertex count **n**, source **s**; imports: **java.util.***.

```
int[] dist = new int[n];
Arrays.fill(dist, -1);
Queue<Integer> q = new ArrayDeque<>();
dist[s] = 0; q.add(s);
while (!q.isEmpty()) {
    int u = q.remove();
    for (int v : g[u]) {
        if (dist[v] != -1) continue;
        dist[v] = dist[u] + 1;
        q.add(v);
    }
}
```

## 09 KEY VARIABLES TO REMEMBER

- **q** - frontier queue; **u, v** - nodes.
- **dist[v]** - shortest hop count from s.
- **s** - source (or use several sources).

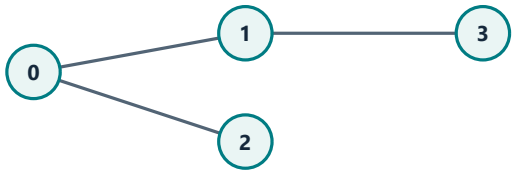
## 10 COMMON MISTAKES

- Marking at dequeue creates duplicate entries.
- Using BFS for arbitrary weighted shortest paths.
- Adding distance again when a node was seen.

## 11 RECOGNITION PATTERN

**If the question says... → Think...**
"Shortest unweighted path" → BFS.
"Rotting / spreading simultaneously" → multi-source BFS.

## 12 ONE TINY EXAMPLE




Queue: [0] → [1,2] → [2,3].
**dist = [0,1,1,2]**; node 3 is two hops away.

# 03 Connected Components

Count separate groups | undirected graph or grid

## 01 WHAT IS IT?

A component is a maximal connected group in an undirected graph. Each fresh DFS/BFS start discovers exactly one component.

## 02 WHEN TO USE

- Number of islands, provinces, networks.
- How many disconnected groups exist?
- Count regions or test if all nodes connect.

## 03 CORE IDEA

1. Initialize visited[] and count = 0.
2. Scan every vertex, including isolated ones.
3. If unseen, increment count; flood-fill it.
4. Continue until the full graph is scanned.

## 04 MEMORY TRICK

**One new start = one new component.**

## 05 IMPORTANT DATA STRUCTURES

- **visited[]** - membership already explored.
- **Stack / queue** - flood-fill the group.
- **count** - number of fresh starts.

## 06 OPERATIONS / ALGORITHM FLOW

START: clear visited; count = 0
↓
Scan next vertex u
↓
Already seen? Skip
↓
Otherwise count++; DFS/BFS from u
↓
Mark every reachable node
↓
END after full scan; return count

## 07 TIME & SPACE COMPLEXITY

| Operation / Case | Time | Space |
|---|---|---|
| All components | O(V + E) | O(V) |
| Grid islands | O(N) | O(N) |

**INTERVIEW: O(V + E) time, O(V) space.** All searches together visit each vertex once.

## 08 CODE SKELETON

Fields: **g**, **n**, fresh **boolean[] seen**. For a grid, scan only valid land cells.

```
void dfs(int u) {
    seen[u] = true;
    for (int v : g[u])
        if (!seen[v]) dfs(v);
}
int countComponents() {
    int count = 0;
    for (int u = 0; u < n; u++) {
        if (seen[u]) continue;
        count++;
        dfs(u);
    }
    return count;
}
```

## 09 KEY VARIABLES TO REMEMBER

- **count** - total components.
- **seen[u]** - u has been assigned to a group.
- **u** - candidate starting vertex.

## 10 COMMON MISTAKES

- Starting DFS once: counts only one group.
- Ignoring isolated vertices (each is a component).
- Treating directed reachability as undirected connectivity; SCCs need a different algorithm.

## 11 RECOGNITION PATTERN

**If the question says... → Think...**
"Number of islands / provinces" → components + DFS/BFS.
"All nodes connected?" → count == 1 (nonempty graph).

## 12 ONE TINY EXAMPLE




Explore 0 → 1: count 1. Explore 2 → 3: count 2. Vertex 4 alone: **count 3**.

# 04 Boundary / Grid Traversal

Boundary-seeded flood fill | surrounded regions, enclaves

## 01 WHAT IS IT?

Treat each grid cell as a vertex and valid moves as edges. For boundary problems, start flood fill from border cells to mark everything connected to the outside.

## 02 WHEN TO USE

- Surrounded regions; land that cannot escape.
- Cells reachable from a boundary / coastline.
- Flood fill with 4- or 8-direction movement.

## 03 CORE IDEA

1. Choose valid cells and movement directions.
2. Seed every valid boundary cell.
3. Check bounds, validity and visited state.
4. Flood-fill; unmarked valid cells are enclosed.

## 04 MEMORY TRICK

**Start outside. Save what touches the border.**

## 05 IMPORTANT DATA STRUCTURES

- **Grid a[][]** - cells / obstacles.
- **safe[][]** - boundary-reachable cells.
- **dr[], dc[]** - neighbor offsets.
- **Stack / queue** - active cells.

## 06 OPERATIONS / ALGORITHM FLOW

START: allocate safe[R][C]
↓
Visit valid cells on all four borders
↓
Mark cell safe; examine 4 neighbors
↓
In bounds, valid, unseen? Recurse
↓
Scan grid after all border fills
↓
END: valid + not safe = enclosed

## 07 TIME & SPACE COMPLEXITY

| Operation / Case | Time | Space |
|---|---|---|
| Boundary scan | O(R + C) | O(1)* |
| Fill + final scan | O(RC) | O(RC) |

**INTERVIEW: O(RC) time, O(RC) space.** *Border scan only. Grid traversal here is flood fill; spiral order uses another pattern.

## 08 CODE SKELETON

Fields: **int[][] a**, **boolean[][] safe**, **R, C > 0**. Cell value 1 is traversable; border loops run in a method.

```
int[] dr = {-1, 1, 0, 0};
int[] dc = {0, 0, -1, 1};
void fill(int r, int col) {
    if (r < 0 || r >= R || col < 0 ||
        col >= C || a[r][col] != 1 ||
        safe[r][col]) return;
    safe[r][col] = true;
    for (int k = 0; k < 4; k++)
        fill(r + dr[k], col + dc[k]);
}
for (int r = 0; r < R; r++) {
    fill(r, 0); fill(r, C - 1);
}
for (int col = 0; col < C; col++) {
    fill(0, col); fill(R - 1, col);
}
```

## 09 KEY VARIABLES TO REMEMBER

- **r, col** - current row and column.
- **R, C** - dimensions; **k** - direction.
- **safe** - connected to boundary.

## 10 COMMON MISTAKES

- Reading a[r][col] before bounds checks.
- Mixing row/column indices or 4/8 directions.
- Skipping a border; forgetting the final scan.

## 11 RECOGNITION PATTERN

**If the question says... → Think...**
"Surrounded / cannot reach edge" → boundary flood fill, then inspect unmarked cells.

## 12 ONE TINY EXAMPLE

| | | | |
|---|---|---|---|
| 1 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 0 | 0 | 0 | 0 |

4-neighbor moves: only (0,0) touches the border. The two middle 1s form an **enclosed region of size 2**.

# 05 Coloring / Bipartite

Two-color constraint | no edge may join equal colors

## 01 WHAT IS IT?

A bipartite graph can be split into two groups with every edge crossing groups. Alternate colors during traversal; a same-color edge is a contradiction (an odd cycle).

## 02 WHEN TO USE

- Divide people into two teams; enemy pairs.
- Check bipartite; avoid adjacent equal groups.
- Two-way constraints or odd-cycle detection.

## 03 CORE IDEA

1. Set all colors to -1 (unassigned).
2. For every uncolored component, seed color 0.
3. Give an unseen neighbor the opposite color.
4. If adjacent colors match, return false.

## 04 MEMORY TRICK

**Neighbors wear opposite colors.**

## 05 IMPORTANT DATA STRUCTURES

- **color[]** - -1 unseen; otherwise 0 or 1.
- **Queue / recursion** - traverse constraints.
- **Adjacency list** - undirected constraints.

## 06 OPERATIONS / ALGORITHM FLOW

START: color[] = -1
↓
Pick uncolored s; color 0; enqueue
↓
Pop u; inspect each neighbor v
↓
Uncolored? Assign 1 - color[u]
↓
Same color? END false
↓
All components checked? END true

## 07 TIME & SPACE COMPLEXITY

| Operation / Case | Time | Space |
|---|---|---|
| Two-color all nodes | O(V + E) | O(V) |

**INTERVIEW: O(V + E) time, O(V) space.** General k-coloring for k > 2 is a different, harder problem.

## 08 CODE SKELETON

Inside a boolean method; initialize **int[] color**. Treat each pair as an undirected edge; **java.util.***.

```
Arrays.fill(color, -1);
Queue<Integer> q = new ArrayDeque<>();
for (int s = 0; s < n; s++) {
    if (color[s] != -1) continue;
    color[s] = 0; q.add(s);
    while (!q.isEmpty()) {
        int u = q.remove();
        for (int v : g[u]) {
            if (color[v] == -1) {
                color[v] = 1 - color[u];
                q.add(v);
            } else if (color[v] == color[u])
                return false;
        }
    }
}
return true;
```

## 09 KEY VARIABLES TO REMEMBER

- **color[v]** - group assignment.
- **1 - color[u]** - opposite group.
- **s** - source of each component.

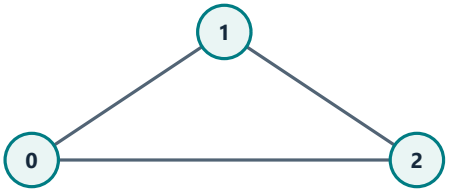
## 10 COMMON MISTAKES

- Only checking the first component.
- Using 0 as both uncolored and a real color.
- Missing self-loops (always a conflict).

## 11 RECOGNITION PATTERN

**If the question says... → Think...**
"Two teams / enemy pairs / bipartite" → two-color BFS/DFS.

## 12 ONE TINY EXAMPLE




0=0, 1=1, 2=0 on a triangle. Edge (2,0) joins color 0 to color 0: **not bipartite**.

# 06 Backtracking in Graphs

Enumerate choices | all simple paths, constrained search

## 01 WHAT IS IT?

Try a choice, recurse, then undo it before trying the next choice. For all simple paths, visited means "on this path" rather than "ever visited."

## 02 WHEN TO USE

- List every valid path / route / assignment.
- Hamiltonian path or constrained small graph.
- Choices must be undone to try alternatives.

## 03 CORE IDEA

1. Add u to the path; mark it inPath.
2. At target, copy the current path to output.
3. Otherwise recurse into legal neighbors.
4. Remove u and unmark it on return.

## 04 MEMORY TRICK

**Choose. Explore. Undo.**

## 05 IMPORTANT DATA STRUCTURES

- **List path** - current candidate route.
- **inPath[]** - forbid cycles on this route.
- **Recursion stack** - outstanding choices.
- **Output list** - copies of completed paths.

## 06 OPERATIONS / ALGORITHM FLOW

START: empty path; inPath[] = false

↓

Choose u; append and mark

↓

u == target? Copy this path

↓

Otherwise try legal neighbors

↓

Undo: remove u; unmark u

↓

END after every branch is explored

## 07 TIME & SPACE COMPLEXITY

| Operation / Case | Time | Space |
|---|---|---|
| Search + copy outputs | O(S + PL) | O(V)* |
| Dense worst case | O(V!) | O(V)* |

**INTERVIEW: Output-sensitive; factorial worst case.** S = neighbor checks, P = paths, L = max length. *Excludes stored output O(PL).

## 08 CODE SKELETON

Fields: **g**, **boolean[] inPath**, **List<Integer> path**, **List<List<Integer>> out**. Simple graph assumed.

```
void paths(int u, int t) {
    inPath[u] = true;
    path.add(u);
    if (u == t) {
        out.add(new ArrayList<>(path));
    } else {
        for (int v : g[u])
            if (!inPath[v]) paths(v, t);
    }
    path.remove(path.size() - 1);
    inPath[u] = false;
}
// Start with empty path / out:
paths(s, t);
```

## 09 KEY VARIABLES TO REMEMBER

- **u, t** - current node and target.
- **inPath** - current recursion branch only.
- **path, out** - working route and copies.

## 10 COMMON MISTAKES

- Global permanent visited hides valid paths.
- Forgetting cleanup on a return branch.
- Storing path itself instead of a fresh copy.
- Calling enumeration O(V + E); outputs can explode.

## 11 RECOGNITION PATTERN

**If the question says... → Think...**
"All possible simple paths" → backtracking.
"Just reachable?" → ordinary DFS/BFS.

## 12 ONE TINY EXAMPLE




Edges 0→1, 0→2, 1→3, 2→3. Output **[0,1,3]**, undo, then **[0,2,3]**.

# 07 Indegree / Kahn

Incoming-edge counts | topological order, dependencies

## 01 WHAT IS IT?

Indegree counts incoming edges in a directed graph. Kahn's algorithm repeatedly removes zero-indegree nodes to produce a topological order of a DAG.

## 02 WHEN TO USE

- Course prerequisites; dependency scheduling.
- Which tasks have no prerequisites left?
- Topological order or directed-cycle detection.

## 03 CORE IDEA

1. For each u → v, increment indeg[v].
2. Enqueue all zero-indegree nodes.
3. Remove u, output it, decrement neighbors.
4. Enqueue newly zero nodes; count removals.

## 04 MEMORY TRICK

**Zero incoming = ready to go.**

## 05 IMPORTANT DATA STRUCTURES

- **indeg[]** - remaining incoming edges.
- **Queue** - currently ready vertices.
- **Order list** - processed sequence.

## 06 OPERATIONS / ALGORITHM FLOW

START: count incoming edges
↓
Enqueue every indegree-0 vertex
↓
Pop u; append u to order
↓
For each u -> v: decrement indeg[v]
↓
If indeg[v] becomes 0, enqueue v
↓
END: order.size < V implies cycle

## 07 TIME & SPACE COMPLEXITY

| Operation / Case | Time | Space |
|---|---|---|
| Count indegrees | O(V + E) | O(V) |
| Kahn + cycle check | O(V + E) | O(V) |

**INTERVIEW: O(V + E) time, O(V) space.** A valid order exists iff the directed graph is acyclic.

## 08 CODE SKELETON

Directed adjacency list **g**; **n** vertices; imports **java.util.***. Order is valid only if cycle is false.

```
int[] indeg = new int[n];
for (int u = 0; u < n; u++)
    for (int v : g[u]) indeg[v]++;
Queue<Integer> q = new ArrayDeque<>();
for (int u = 0; u < n; u++)
    if (indeg[u] == 0) q.add(u);
List<Integer> order = new ArrayList<>();
while (!q.isEmpty()) {
    int u = q.remove();
    order.add(u);
    for (int v : g[u])
        if (--indeg[v] == 0) q.add(v);
}
boolean cycle = order.size() < n;
```

## 09 KEY VARIABLES TO REMEMBER

- **indeg[v]** - prerequisites still pending.
- **q** - ready tasks; **order** - removal order.
- **cycle** - fewer than n nodes removed.

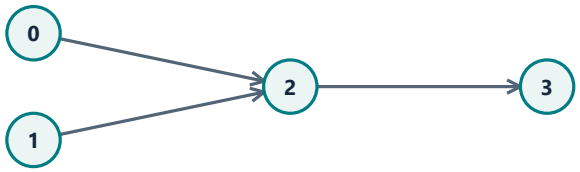
## 10 COMMON MISTAKES

- Reversing prerequisite edges.
- Enqueueing before indegree becomes zero.
- Returning a partial order despite a cycle.
- Reusing indeg after Kahn has modified it.

## 11 RECOGNITION PATTERN

**If the question says... → Think...**
"Prerequisites / valid order" → Kahn.
"No incoming edges" → indegree == 0.

## 12 ONE TINY EXAMPLE




0→2, 1→2, 2→3: indeg=[0,0,2,1]. Remove 0,1; then 2; then 3: **valid order [0,1,2,3]**.

# 08 Union Find / DSU

Disjoint Set Union | connectivity under merges

## 01 WHAT IS IT?

Maintain disjoint groups using a representative root. Find retrieves the root; union merges roots using size/rank, with path compression speeding later finds.

## 02 WHEN TO USE

- Repeated merge / same-group queries.
- Redundant undirected edge; Kruskal MST.
- Connectivity when edges are only added.

## 03 CORE IDEA

1. Initially each item is its own root.
2. Find roots; compress paths along the way.
3. If roots match, the groups already connect.
4. Attach smaller root under larger; update size.

## 04 MEMORY TRICK

**Find leader. Merge leaders.**

## 05 IMPORTANT DATA STRUCTURES

- **parent[]** - parent pointers; root points to itself.
- **size[]** - group size at each root.
- **rank[]** - alternative balancing heuristic; choose size OR rank.

## 06 OPERATIONS / ALGORITHM FLOW

START: parent[i] = i; size[i] = 1
↓
find(a), find(b): compress paths
↓
Same root? Return false (no merge)
↓
Choose larger set as new root
↓
Attach smaller root; add sizes
↓
END: return true for successful merge

## 07 TIME & SPACE COMPLEXITY

| Operation / Case | Time | Space |
|---|---|---|
| Initialize N items | O(N) | O(N) |
| Find / union | O(alpha(N))* | O(1) extra |

**INTERVIEW: Amortized O(alpha(N)) per operation; O(N) storage.** *Not worst-case O(1); iterative code avoids recursion space.

## 08 CODE SKELETON

Fields: **int[] parent, size**. Before use: for each i, set **parent[i]=i**, **size[i]=1**. alpha is inverse Ackermann.

```
int find(int x) {
    while (x != parent[x]) {
        parent[x] = parent[parent[x]];
        x = parent[x];
    }
    return x;
}
boolean union(int a, int b) {
    a = find(a); b = find(b);
    if (a == b) return false;
    if (size[a] < size[b]) {
        int t = a; a = b; b = t;
    }
    parent[b] = a;
    size[a] += size[b];
    return true;
}
```

## 09 KEY VARIABLES TO REMEMBER

- **a, b** - roots after find.
- **parent[x]** - current parent / leader.
- **size[root]** - valid size only at roots.

## 10 COMMON MISTAKES

- Merging original nodes rather than their roots.
- Updating size on a non-root.
- Using DSU for directed reachability.
- Ordinary DSU cannot handle edge deletions.

## 11 RECOGNITION PATTERN

**If the question says... → Think...**
"Merge groups / are they connected?" → DSU.
"Undirected redundant edge" → union returns false.

## 12 ONE TINY EXAMPLE

**{0} {1} {2} {3} → {0,1} {2,3}**
**→ {0,1,2,3}**

union(0,1), union(2,3), union(1,2). All four share one root. **union(0,3) returns false**.

# 09 Dijkstra

Shortest weighted paths | all edge weights must be nonnegative

## 01 WHAT IS IT?

Keep tentative distances and repeatedly process the cheapest current entry. Relax outgoing edges; a stale priority-queue entry is skipped.

## 02 WHEN TO USE

- Minimum total cost / time with weights >= 0.
- Weighted graph; shortest path from a source.
- Grid moves with nonnegative varying costs.

## 03 CORE IDEA

1. Set distances to INF; source to 0.
2. Push (0, source) into a min-heap.
3. Pop (d,u); skip if d != dist[u].
4. Improve neighbors and push new entries.

## 04 MEMORY TRICK

**Cheapest available distance first.**

## 05 IMPORTANT DATA STRUCTURES

- **dist[]** (long) - best known costs.
- **PriorityQueue** - smallest tentative cost.
- **Weighted adjacency list** - (to, weight).
- **parent[]** (optional) - shortest-path tree.

## 06 OPERATIONS / ALGORITHM FLOW

START: dist[] = INF; dist[s] = 0
↓
Push (0,s) into min-heap
↓
Pop cheapest (d,u); skip if stale
↓
For each edge u -> v of weight w
↓
d + w < dist[v]? Update and push
↓
END when heap empty (or target finalized)

## 07 TIME & SPACE COMPLEXITY

| Operation / Case | Time | Space |
|---|---|---|
| Lazy heap | O(V + E log(E + 2)) | O(V + E) |
| Simple graph | O((V + E) log V) | O(V + E) |

**INTERVIEW: O((V + E) log V)** for a simple graph. Lazy duplicates use **O(V + E)** space; indexed decrease-key heaps use O(V).

## 08 CODE SKELETON

Java 17+; **List<Edge>[] g**; **java.util.***. Weights are nonnegative ints; finite answers must be < INF.

```
record Edge(int to, int w) {}
record State(int u, long d) {}
long INF = Long.MAX_VALUE / 4;
long[] dist = new long[n];
Arrays.fill(dist, INF); dist[s] = 0;
PriorityQueue<State> pq =
    new PriorityQueue<>(
        Comparator.comparingLong(State::d));
pq.add(new State(s, 0));
while (!pq.isEmpty()) {
    State cur = pq.remove();
    int u = cur.u(); long d = cur.d();
    if (d != dist[u]) continue;
    for (Edge e : g[u]) {
        long nd = d + e.w();
        if (nd < dist[e.to()]) {
            dist[e.to()] = nd;
            pq.add(new State(e.to(), nd));
        }
    }
}
```

## 09 KEY VARIABLES TO REMEMBER

- **d** - popped cost; **nd** - candidate cost.
- **dist[v]** - best cost; **INF** - unreachable.

## 10 COMMON MISTAKES

- Negative edges or arbitrary longest paths.
- Ignoring stale entries; int sum overflow.
- Comparator by subtraction can overflow.

## 11 RECOGNITION PATTERN

**If the question says... → Think...**
"Weighted shortest + nonnegative" → Dijkstra.
"Maximum sum path" → NOT generic Dijkstra; DAG longest paths use topological DP.

## 12 ONE TINY EXAMPLE




0→1 (4), 0→2 (1), 2→1 (2). Pop 0,2,1: **dist[1]=3**. Later entry (4,1) is stale.

# 10 Tarjan / Low-Link

Undirected bridges + articulation points | DFS discovery and escape routes

## 01 WHAT IS IT?

Track DFS discovery times and the earliest discovery reachable through a subtree and a back edge. Low-link tests reveal edges or vertices whose removal disconnects the graph.

## 02 WHEN TO USE

- Critical connections / bridges.
- Cut vertices / single points of failure.
- Directed SCCs: use Tarjan's stack variant.

## 03 CORE IDEA

1. Set disc[u] = low[u] = timer++.
2. DFS child v; merge low[v] into low[u].
3. Back edge to v: merge disc[v], not low[v].
4. Bridge: low[v] > disc[u]; cut: >= (non-root).

## 04 MEMORY TRICK

**Can my child reach above me?**

## 05 IMPORTANT DATA STRUCTURES

- **disc[]** - first discovery time; -1 unseen.
- **low[]** - earliest reachable discovery.
- **Edge IDs** - skip only the entering edge.
- **cut[], bridges** - recorded answers.

## 06 OPERATIONS / ALGORITHM FLOW

START: disc[] = -1; timer = 0

↓

DFS every unseen component

↓

Tree edge: recurse; merge low[child]

↓

Back edge: merge disc[neighbor]

↓

Test bridge >; non-root cut >=

↓

END: root cut iff > 1 DFS child

## 07 TIME & SPACE COMPLEXITY

| Operation / Case | Time | Space |
|---|---|---|
| Bridges / cut vertices | O(V + E) | O(V)* |

**INTERVIEW: O(V + E) time, O(V) working space.** *Stored bridge output adds O(E); adjacency excluded.

## 08 CODE SKELETON

Java 17+. Fields: **g, disc, low, timer, cut, bridges**. disc starts -1, timer 0. Both copies of an undirected edge share its ID; driver loop runs in a method.

```
record Edge(int to, int id) {}
void dfs(int u, int pe) {
    disc[u] = low[u] = timer++;
    int kids = 0;
    for (Edge e : g[u]) {
        if (e.id() == pe) continue;
        int v = e.to();
        if (disc[v] == -1) {
            kids++; dfs(v, e.id());
            low[u] = Math.min(low[u], low[v]);
            if (low[v] > disc[u])
                bridges.add(new int[]{u, v});
            if (pe != -1 && low[v] >= disc[u])
                cut[u] = true;
        } else {
            low[u] = Math.min(low[u], disc[v]);
        }
    }
    if (pe == -1 && kids > 1) cut[u] = true;
}
for (int u = 0; u < n; u++)
    if (disc[u] == -1) dfs(u, -1);
```

## 09 KEY VARIABLES TO REMEMBER

- **disc, low** - discovery / earliest escape.
- **pe** - parent edge ID; **kids** - DFS children.
- **timer** - increasing discovery counter.

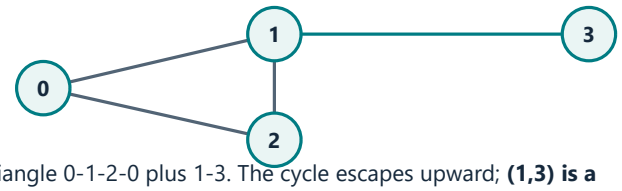
## 10 COMMON MISTAKES

- Mixing bridge > with articulation >=.
- Skipping parent vertex loses parallel back edges.
- SCC variant requires onStack[] + stack; at low[u]==disc[u], pop an SCC. Different code.

## 11 RECOGNITION PATTERN

**If the question says... → Think...**
"Critical edge" → low[child] > disc[parent].
"Critical node" → cut test + root special case.

## 12 ONE TINY EXAMPLE




Triangle 0-1-2-0 plus 1-3. The cycle escapes upward; **(1,3) is a bridge**, and **1 is a cut vertex**.

# 11 Component Labeling

Store region IDs and sizes | repeated component queries

## 01 WHAT IS IT?

Assign the same integer ID to every cell or vertex in a connected region. Labels turn later "same region?" and region-size questions into constant-time lookups.

## 02 WHEN TO USE

- Largest island after changing a 0 to 1.
- Repeated same-region or region-size queries.
- Need IDs and sizes, rather than only a count.

## 03 CORE IDEA

1. Scan cells; find unlabeled valid land.
2. Allocate a new ID; flood-fill with that ID.
3. Accumulate and store the region size.
4. Use labels; deduplicate IDs when merging.

## 04 MEMORY TRICK

**Paint a region. Give it a name.**

## 05 IMPORTANT DATA STRUCTURES

- **id[][]** - region label; 0 = unlabeled.
- **size[]** - number of cells for each ID.
- **Stack / queue** - flood-fill cells.
- **Set of IDs** (merge queries) - avoid double counting.

## 06 OPERATIONS / ALGORITHM FLOW

START: id[][] = 0; component = 0

↓

Scan valid, unlabeled cell

↓

component++; flood-fill this ID

↓

Count cells; store size[component]

↓

Continue scan; answer queries by ID

↓

END: labels + sizes ready

## 07 TIME & SPACE COMPLEXITY

| Operation / Case | Time | Space |
|---|---|---|
| Grid preprocessing | O(RC) | O(RC) |
| Same ID / size query | O(1) | O(1) |

**INTERVIEW: O(N) preprocessing, O(N) space; O(1) lookup.** Graph labeling instead costs O(V + E) time.

## 08 CODE SKELETON

Fields: **a, id, R, C**; id starts 0. Allocate **size** of length RC+1. Scan runs in a method.

```
int[] dr = {-1, 1, 0, 0};
int[] dc = {0, 0, -1, 1};
int label(int r, int col, int k) {
    if (r < 0 || r >= R || col < 0 ||
        col >= C || a[r][col] != 1 ||
        id[r][col] != 0) return 0;
    id[r][col] = k;
    int total = 1;
    for (int d = 0; d < 4; d++)
        total += label(r + dr[d],
                       col + dc[d], k);
    return total;
}
int k = 0;
for (int r = 0; r < R; r++)
    for (int col = 0; col < C; col++)
        if (a[r][col] == 1 && id[r][col] == 0)
            size[++k] = label(r, col, k);
```

## 09 KEY VARIABLES TO REMEMBER

- **k** - region ID starting at 1.
- **id[r][col]** - cell's component label.
- **total, size[k]** - local / stored cell count.

## 10 COMMON MISTAKES

- Reusing 0 for both land labels and unseen.
- Counting a neighboring component twice.
- Labeling water or using wrong connectivity.

## 11 RECOGNITION PATTERN

**If the question says... → Think...**
"Region ID / repeated size queries" → label components.
"Flip one cell" → 1 + sizes of distinct neighbor IDs.

## 12 ONE TINY EXAMPLE

| | | |
|---|---|---|
| 1 | 1 | 0 |
| 0 | 0 | 0 |
| 2 | 0 | 3 |

Before: 110 / 000 / 101. Labels: **110 / 000 / 203**; sizes: **1→2, 2→1, 3→1**.

# 12 Reverse Graph Traversal

Reverse every directed edge | who can reach a target?

## 01 WHAT IS IT?

Replace each directed edge u → v with v → u. A traversal from target t in the reversed graph finds exactly the original vertices that can reach t.

## 02 WHEN TO USE

- Which nodes can reach a destination?
- Many source queries for one fixed target.
- Reverse shortest paths; Kosaraju SCC pass.

## 03 CORE IDEA

1. Build rev[v].add(u) for each u → v.
2. Start DFS/BFS from the target in rev.
3. Mark every reached predecessor.
4. Use marks for original reach-to-target queries.

## 04 MEMORY TRICK

**To find who reaches me, walk backward.**

## 05 IMPORTANT DATA STRUCTURES

- **rev[]** - incoming neighbors of original graph.
- **can[]** - can reach target in original graph.
- **Queue / stack** - reverse traversal.

## 06 OPERATIONS / ALGORITHM FLOW

START: allocate empty rev lists
↓
For each u -> v, add v -> u
↓
Mark target t; enqueue t
↓
Pop u; traverse rev[u]
↓
Mark and enqueue unseen predecessors
↓
END: can[x] answers x -> ... -> t

## 07 TIME & SPACE COMPLEXITY

| Operation / Case | Time | Space |
|---|---|---|
| Build reversed graph | O(V + E) | O(V + E) |
| Traverse + marks | O(V + E) | O(V) |

**INTERVIEW: O(V + E) time; O(V + E) space including rev.**
Traversal alone adds O(V). Repeated reachability queries: O(1).

## 08 CODE SKELETON

Directed **g**; target **t**; **java.util.***. Generic array creation has an unchecked warning. For weighted graphs, preserve weights and use Dijkstra.

```
List<Integer>[] rev = new List[n];
for (int u = 0; u < n; u++)
    rev[u] = new ArrayList<>();
for (int u = 0; u < n; u++)
    for (int v : g[u]) rev[v].add(u);
boolean[] can = new boolean[n];
Queue<Integer> q = new ArrayDeque<>();
can[t] = true; q.add(t);
while (!q.isEmpty()) {
    int u = q.remove();
    for (int v : rev[u]) {
        if (can[v]) continue;
        can[v] = true; q.add(v);
    }
}
```

## 09 KEY VARIABLES TO REMEMBER

- **rev[v]** - original predecessors of v.
- **t** - fixed target; **can[x]** - x reaches t.
- **q** - active reverse frontier.

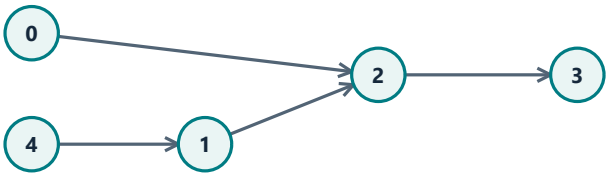
## 10 COMMON MISTAKES

- Traversing forward from t answers a different question.
- Ignoring edge direction or losing weights.
- One reverse DFS alone does not compute SCCs; Kosaraju needs ordering and a second pass.

## 11 RECOGNITION PATTERN

**If the question says... → Think...**
"All nodes that can reach t" → reverse + DFS/BFS.
"Distances to one target" → reverse + BFS/Dijkstra.

## 12 ONE TINY EXAMPLE




Original: 0→2, 1→2, 2→3, 4→1. Reverse BFS from 3: **3,2,0,1,4**; all can reach 3.

# Master Revision / 1 of 2

All 12 patterns at a glance | match the clue, recall the structure, state the cost

| Algorithm | When to Use | Main DS | Time | Memory Trick |
|---|---|---|---|---|
| **DFS** / p1 | Reachability; deep traversal | Stack + seen | O(V + E) | Go deep; return |
| **BFS** / p2 | Shortest unweighted; levels | Queue + dist | O(V + E) | Level by level |
| **Components** / p3 | Count disconnected groups | DFS/BFS + seen | O(V + E) | Fresh start = group |
| **Boundary / grid** / p4 | Enclaves; outside reach | Grid + safe + stack | O(RC) | Save the border |
| **Bipartite** / p5 | Two groups; enemy pairs | color + queue | O(V + E) | Opposite colors |
| **Backtracking** / p6 | All simple paths; choices | path + inPath | O(S + PL); factorial worst case | Choose, explore, undo |
| **Indegree / Kahn** / p7 | DAG task order; cycle test | indeg + queue | O(V + E) | Zero means ready |
| **DSU** / p8 | Merge; online connectivity | parent + size | O(alpha(N)) amortized / op | Find + merge leaders |
| **Dijkstra** / p9 | Shortest weighted; w >= 0 | dist + min-heap | O((V+E) log V)* | Cheapest first |
| **Tarjan low-link** / p10 | Bridges; articulation points | disc + low | O(V + E) | Can child escape? |
| **Labeling** / p11 | IDs / sizes; repeated queries | id + size | O(N) grid prep; O(1) query | Paint + name |
| **Reverse graph** / p12 | Who can reach target t? | rev + seen | O(V + E) | Walk backward |

**Complexity contract:** adjacency lists unless stated. N = RC for grid pages; N = number of DSU items for p8. S = neighbor checks, P = output paths, L = maximum path length for p6. *Dijkstra bound assumes a simple graph; lazy-heap multigraph bound is O(V + E log(E+2)).

## 01 SPACE YOU SHOULD SAY ALOUD

**Usually O(V):** DFS, BFS, components, coloring, Kahn and Tarjan working state. **Grid:** O(RC). **DSU:** O(N). **Lazy Dijkstra:** O(V + E), including duplicate heap entries. **Reverse:** O(V + E) for the new graph. **Backtracking:** O(V) working state + O(PL) stored output.

## 02 THREE DISTINCTIONS THAT PREVENT WRONG ANSWERS

- **Count vs label:** components counts groups; labeling also saves IDs and sizes.
- **Traversal vs enumeration:** DFS keeps seen permanently; backtracking undoes inPath.
- **Connectivity vs reachability:** DSU / undirected components do not answer directed reachability.

*Auxiliary space excludes input adjacency storage O(V + E) and explicitly excludes outputs where noted. Recursive depth is counted. Each code block is a core template, not a standalone program.

# Master Revision / 2 of 2

Question clue -> algorithm | final interview checks and invariant reminders

| If the question says... | Think... |
|---|---|
| Can A reach B? Explore reachable vertices. | **DFS / BFS** |
| Minimum hops / unweighted shortest path. | **BFS**; first discovery fixes distance |
| Many starting cells spread at once. | **Multi-source BFS**; all sources start at 0 |
| Islands / provinces / separate regions. | **Connected components + DFS/BFS** |
| Surrounded / enclaves / escape to boundary. | **Boundary-seeded grid flood fill** |
| Two teams / enemies / odd-cycle check. | **Bipartite coloring** |
| Enumerate every simple path / choice. | **Backtracking**; undo state |
| Prerequisites / valid task order / DAG. | **Indegree + Kahn topological sort** |
| Repeated group merges / added edges. | **DSU**; find roots, then union |
| Shortest weighted path, all weights >= 0. | **Dijkstra**; skip stale heap entries |
| Critical connection / bridge / cut vertex. | **Tarjan low-link**; > for bridge, >= for cut |
| Region IDs / sizes / flip one grid cell. | **Component labeling**; unique neighbor IDs |
| Who can reach a fixed target? | **Reverse graph + DFS/BFS** |
| Mutually reachable groups in a directed graph. | **SCC**: Tarjan stack variant / Kosaraju |
| Negative weights / longest weighted path. | **Re-check assumptions**; not generic Dijkstra |

## 01 30-SECOND JAVA + INVARIANT CHECK

- **Visited:** DFS marks before recursion; BFS marks before enqueue; backtracking unmarks on return.
- **Sentinels:** dist=-1 for BFS; color=-1; disc=-1; component ID=0 is unseen. Reset between runs.
- **Coverage:** scan all components; decide 4 vs 8 grid neighbors; check bounds first.
- **Java:** ArrayDeque for queues; long for weighted costs; compare with comparingLong. Recursive DFS can overflow on long chains. Java 17+ records appear on p9-p10.

## 02 BOUNDARY CASES TO SAY BEFORE CODING

Isolated / disconnected nodes? Directed or undirected? Self-loops / parallel edges? Negative / zero weights? Empty grid? One source or many? Need a count, one path, all paths, or reusable labels?

**Recall anchors:** BFS dist[v]=dist[u]+1 | DSU find(a)==find(b) | Kahn removed==V
Dijkstra candidate=d+w | bridge low[v]>disc[u] | cut low[v]>=disc[u] (non-root)

**Technical cross-checks:** Princeton Algorithms, Undirected Graphs, Directed Graphs, and Shortest Paths. Skeletons and examples are compact revision versions. For negative edges, consider Bellman-Ford; on a DAG, topological DP handles shortest or longest paths.